Machine Learning

Regression

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

dataset= pd.read_csv('./datasets/petrol_consumption.csv')
print(dataset.head()); print(dataset.shape)

X =dataset[['Petrol_tax', 'Average_income', 'Paved_Highways', 'Population_Driver_licence(%)']]
Y= dataset['Petrol_Consumption']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)

reg_model= LinearRegression()
reg_model.fit(X_train, Y_train)
print(reg_model.coef_, reg_model.intercept_)

Y_pred= reg_model.predict(X_test)
df= pd.DataFrame({'Actual':Y_test, 'Predicted':Y_pred})
print(df)

print('Mean Squared Error:', mean_squared_error(Y_test, Y_pred))
print('Mean Absolute Error:', mean_absolute_error(Y_test, Y_pred))
print('R² score(coefficient of determination):', r2_score(Y_test, Y_pred))
```

Classification

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report

dataset= pd.read_csv('./datasets/bill_authentication.csv')
print(dataset.head()); print(dataset.shape)

X= dataset.drop('Class',axis='columns')
Y= dataset['Class']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)

classifier= LogisticRegression()
classifier.fit(X_train, Y_train)

Y_pred=classifier.predict(X_test)
df= pd.DataFrame({'Actual':Y_test, 'Predicted':Y_pred})
print(df.head())

print(confusion_matrix(Y_test, Y_pred))
print(classification_report(Y_test, Y_pred))
```

Clustering

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

X=pd.read_csv('./datasets/clustering.csv', header=None)

clusters=5
clst_model=KMeans(n_clusters=clusters, random_state=0)
clst_model.fit(X)
print(clst_model.cluster_centers_); print(clst_model.labels_)

N=np.array(X)
plt.scatter(N[:,0], N[:,1], c='blue')
plt.scatter(
    clst_model.cluster_centers_[:,0], clst_model.cluster_centers_[:,1],
    marker='x', c='red')
plt.show()

plt.scatter(N[:,0], N[:,1], c=clst_model.labels_, cmap='jet')
plt.scatter(
    clst_model.cluster_centers_[:,0], clst_model.cluster_centers_[:,1],
    marker='x', c=range(clusters), cmap='jet')
plt.show()
```

Graph Search

Strategies

```
def select_node(strategy, fringe):
    if strategy == 'DFS':    return -1
    if strategy == 'BFS':    return  0
    if strategy == 'UCS':    return get_min('cost',fringe)
    if strategy == 'Greedy': return get_min('heuristic',fringe)
    if strategy == 'Astar':  return get_min('f',fringe)

def get_min(key, fringe):
    idx_min= 0
    for i in range(1, len(fringe)):
        if fringe[i][key] < fringe[idx_min][key] :
            idx_min= i
    return idx_min

def solve(strategy, problem, initial_state):
    fringe=[]; visited=[]
    initial_node= init_node(strategy, problem, initial_state)
    fringe.append(initial_node)
    while len(fringe)>0:
        selected_node= select_node(strategy, fringe)
        current_node= fringe.pop(selected_node)
        if current_node['state'] in visited: continue
        visited.append(current_node['state'])
        if problem.isgoal(current_node['state']):
            return get_solution(current_node, len(visited))
        possible_actions= problem.get_actions(current_node['state'])
        for action in possible_actions:
            next_node= add_node(strategy, problem, current_node, action)
            fringe.append(next_node)
    return None
```

Puzzle8 Problem

```
class Puzzle8:

    def get_empty(self, puzzle):
        for i in range(len(puzzle)):
            if puzzle[i]==0: return i
        return None

    def get_actions(self, puzzle):
        empty= self.get_empty(puzzle)
        possible_actions= []
        if empty not in [0,1,2]: possible_actions.append('^')
        if empty not in [6,7,8]: possible_actions.append('v')
        if empty not in [2,5,8]: possible_actions.append('>')
        if empty not in [0,3,6]: possible_actions.append('<')
        return possible_actions

    def get_state(self, puzzle, action):
        new_puzzle= puzzle[:]
        empty=self.get_empty(new_puzzle)
        if action=='>':
            new_puzzle[empty], new_puzzle[empty+1] = new_puzzle[empty+1], new_puzzle[empty]
        if action=='<':
            new_puzzle[empty], new_puzzle[empty-1] = new_puzzle[empty-1], new_puzzle[empty]
        if action=='^':
            new_puzzle[empty], new_puzzle[empty-3] = new_puzzle[empty-3], new_puzzle[empty]
        if action=='v':
            new_puzzle[empty], new_puzzle[empty+3] = new_puzzle[empty+3], new_puzzle[empty]
        return new_puzzle

    def isgoal(self, puzzle):
        for i in range(len(puzzle)):
            if puzzle[i]!=i: return False
        return True

    def compute_cost(self, puzzle, action):
        return 1

    def compute_heuristic(self, puzzle):
        estimated_cost=0
        for i in range(len(puzzle)):
            if puzzle[i]!=i : estimated_cost+=1
        return estimated_cost
```

`

`

`

Maze Problem

```
class Maze:

    def __init__(self):
        #initialize maze, start_position and goal_position
        self.maze= maze or [
            [" ", "G", " ", " ", " ", " "],
            [" ", "#", "#", " ", " ", " "],
            [" ", " ", "#", " ", " ", " "],
            ["#", " ", "#", "#", "#", " "],
            [" ", " ", " ", " ", " ", " "],
            ["S", "#", " ", " ", " ", " "],
        ]
        self.start_position= (5,0)
        self.goal_position= (0,1)

    def get_actions(self, state):
        row, col= state["position"][0], state["position"][1]
        n_rows, n_cols= len(self.maze), len(self.maze[0])
        possible_actions= []
        if (row-1)>=0     and self.maze[row-1][ col ]!="#": possible_actions.append('^')
        if (row+1)<n_rows and self.maze[row+1][ col ]!="#": possible_actions.append('v')
        if (col+1)<n_cols and self.maze[ row ][col+1]!="#": possible_actions.append('>')
        if (col-1)>=0     and self.maze[ row ][col-1]!="#": possible_actions.append('<')
        return possible_actions

    def get_state(self, state, action):
        row, col= state["position"][0], state["position"][1]
        if action=='^': row-=1
        if action=='v': row+=1
        if action=='>': col+=1
        if action=='<': col-=1
        new_state= {}
        new_state["position"]= (row, col)
        new_state["direction"]= action
        return new_state

    def isgoal(self, state):
        return state["position"]==self.goal_position

    def compute_cost(self, state, action):
        direction= state["direction"]
        if direction==action:
            return 1
        return 5

    def compute_heuristic(self, state):
        return (
            abs(self.goal_position[0]-state["position"][0]) +
            abs(self.goal_position[1]-state["position"][1])
        )
```

Evolutionary Algorithms

Genetic Algorithm

```
def knapsack_fitness(genome, items, weight_limit):
    total_value, total_weight = 0, 0
    for i in range(len(items)):
        if genome[i] == 1:
            total_value += items[i]['value']
            total_weight += items[i]['weight']
    if total_weight > weight_limit:
        return 0
    return total_value

def population_fitness(population, items, weight_limit):
    return [knapsack_fitness(genome, items, weight_limit) for genome in population]

def single_point_crossover(genome_a, genome_b):
    p = randrange(len(genome_a))
    return \
        genome_a[0:p] + genome_b[p:], \
        genome_b[0:p] + genome_a[p:]

def mutation(genome):
    p = randrange(len(genome))
    if random() > 0.5:
        genome[p] = abs(genome[p] - 1)
    return genome

def selection_pair(population, scores):
    return choices(
        population=population,
        k=2,
        weights=scores if sum(scores)!=0 else None,
    )

def run_evolution(items, weight_limit, population_size, generation_limit):
    genome_length=len(items)
    population = generate_population(population_size, genome_length)
    scores = population_fitness(population, items, weight_limit)
    population, scores = sort_by_fitness(population, scores)
    for i in range(generation_limit):
        next_population = [] + population[0:2]
        for j in range(int(len(population) / 2) - 1):
            parents = selection_pair(population, scores)
            offspring_a, offspring_b = single_point_crossover(parents[0], parents[1])
            offspring_a, offspring_b = mutation(offspring_a), mutation(offspring_b)
            next_population += [offspring_a, offspring_b]
        population = next_population
        scores = population_fitness(population, items, weight_limit)
        population, scores = sort_by_fitness(population, scores)
    return population, scores
```

`

`

`

`

`

`

`

`

`

`

`

`

`

`

`

`

`

`

`

`

`

Knowledge Representation

Students

```
from pytholog import KnowledgeBase, Expr
knowledge_base = KnowledgeBase("students")
knowledge_base([
    "teach(hesham,ai)",
    "teach(mostafa,programming)",
    "learn(mohammed,ai)",
    "learn(mohammed,programming)",
    "learn(ahmed,ai)",
    "student(X) :- learn(X,Y)",
    "teacher(X) :- teach(X,Y)",
    "course(X) :- learn(Y,X)",
    "course(X) :- teach(Y,X)",
])

question= Expr("learn(mohammed, X)")
answer= knowledge_base.query(question)
print(answer)

question= Expr("learn(X,ai)")
answer= knowledge_base.query(question)
print(answer)

question= Expr("teach(X, ai)")
answer= knowledge_base.query(question)
print(answer)
```

Color Map


```
from pytholog import KnowledgeBase, Expr
knowledge_base = KnowledgeBase("Colormap")
knowledge_base([
    "different(red, green)",
    "different(red, blue)",
    "different(green, red)",
    "different(green, blue)",
    "different(blue, red)",
    "different(blue, green)",
    "coloring(A, M, G, T, F) :- different(M,T), different(M,A), different(A,T),different(A,M), different(A,G), different(A,F),
different(G,F), different(G,T)"
])

question= Expr("coloring(Alabama, Mississippi, Georgia, Tennessee, Florida)")
answer= knowledge_base.query(question, cut=True)
print(answer)
```

`

`

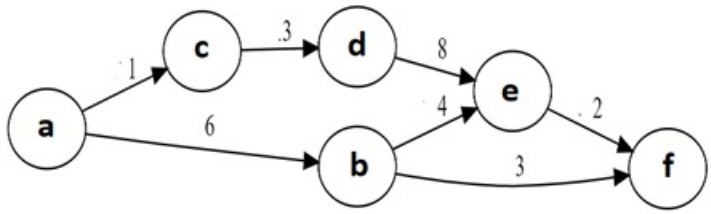
`

`

`

`

Graph




```
from pytholog import KnowledgeBase, Expr
knowledge_base = KnowledgeBase("Graph")
knowledge_base([
    "edge(a, b, 6)",
    "edge(a, c, 1)",
    "edge(b, e, 4)",
    "edge(b, f, 3)",
    "edge(c, d, 3)",
    "edge(d, e, 8)",
    "edge(e, f, 2)",
    "path(X, Y, W) :- edge(X , Y, W)",
    "path(X, Y, W) :- edge(X, Z, W1), path(Z, Y, W2), W is W1 + W2",
])

question= Expr("path(a, f, W)")
answer= knowledge_base.query(question)
print(answer)
```

Symptoms

```
from pytholog import KnowledgeBase, Expr
knowledge_base = KnowledgeBase("Symptoms")
knowledge_base([
    "has_symptom(flu, fever)",
    "has_symptom(flu, headache)",
    "has_symptom(flu, body_aches)",
    "has_symptom(flu, cough)",
    "has_symptom(flu, sore_throat)",
    "has_symptom(flu, runny_nose)",
    "has_symptom(allergy, sneezing)",
    "has_symptom(allergy, watery_eyes)",
    "has_symptom(allergy, runny_nose)",
    "has_symptom(allergy, itchy_eyes)",
    "has_symptom(cold, sneezing)",
    "has_symptom(cold, watery_eyes)",
    "has_symptom(cold, runny_nose)",
    "has_symptom(cold, cough)",
    "has_symptom(cold, sore_throat)",
])

question= Expr("has_symptom(cold, X)")
answer= knowledge_base.query(question)
print(answer)

question= Expr("has_symptom(X, cough)")
answer= knowledge_base.query(question)
print(answer)
```